# Where the compile-time guarantee stops

Rust eliminates data races, not race conditions. Aliasing errors, not lock-ordering errors

## INSIDE THE GUARANTEE

Rejected by the compiler, on every build, covering every execution of the program

### Lifetime and aliasing

enforced by ownership and borrowing

- Use-after-free
- Double-free
- Iterator invalidation

### Unsynchronised access to a location

enforced by Send/Sync: these are *data races*

- Data race
- Unguarded shared mutation

In C++ both compile. Only ThreadSanitizer reports them.

**No test coverage required for the guarantee to hold**

## OUTSIDE THE GUARANTEE

Compiles in safe Rust and misbehaves, exactly as its C++ counterpart does

### Order of lock acquisition

not prevented by either compiler

- Lock-order deadlock: hangs

The borrow checker tracks a guard’s lifetime perfectly. That says nothing about the order threads take locks.

### Atomicity of a composed operation

not prevented either: these are *race conditions*

- Lost update: silently wrong value

Every access is correctly synchronised, so there is no race to find: no tool in either toolchain reports it.

**Fails quietly: a hang or a corrupted counter, not a crash**

**the boundary**

The phrase “thread safety” spans both columns. Rust’s guarantee covers only the left one.

It retires one category of concurrency defect entirely and leaves the other in place, and the one left in place fails silently.